(1)

Q1) What is true about Interpeter?

- Direct execute of operations in Sourc program.

Q2) Which Phase does not deal with Sourc program directly?

- Code optimization

Q3) Write the Regular expression for String that generates string of all 1's and 0's followed by four zero's.

- (01)*0000

Q4) The _____ and _____ deals with large fraction of errors.

- Syntax analysis and Semantics analysis

Q5) What is Not true about lexical analyzer?

- Use error recovery and fix all errors which prevent Syntax errors

Q6) Which phase that generate executable and absolute machine code?

- linker

---

(2) Write lexemes and Tokens, and which tokens inserted to symbol table?

```
// comment
public int addNumbers ( int a, float b)
```

lexeme	Token	Symbol table (yes, no)	...	...	...
public	public	NO	a	ID	Yes
int	int	NO	,	comma	NO
addNumbers	Identi	Yes	float	float	No
(	L-Parenthesis	No	b	ID	Yes
int	int	NO	)	R-parenthesis	NO

(3) Write 4-tuple of the following grammar:

$E \rightarrow S; \mid \varepsilon$

$S \rightarrow$ if T D

$D \rightarrow a \mid b \mid D = N$

$T \rightarrow$ true | false

$N \rightarrow 0 \mid 1$

Answer :- $G = (V, \Sigma, R, S)$

$V = \{E, S, D, T, N\}$

$S = \{E\}$

$\Sigma = \{;, \text{if}, a, b, =, \text{true}, \text{false}, 0, 1\}$

$R =$ Whole grammar

- Derive Rightmost

$E \rightarrow S;$

$\rightarrow$ if T D ;

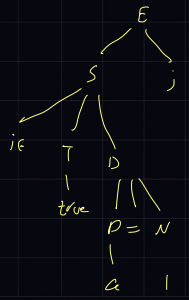
$\rightarrow$ if T $D = N$

$\rightarrow$ if T $D = 1$

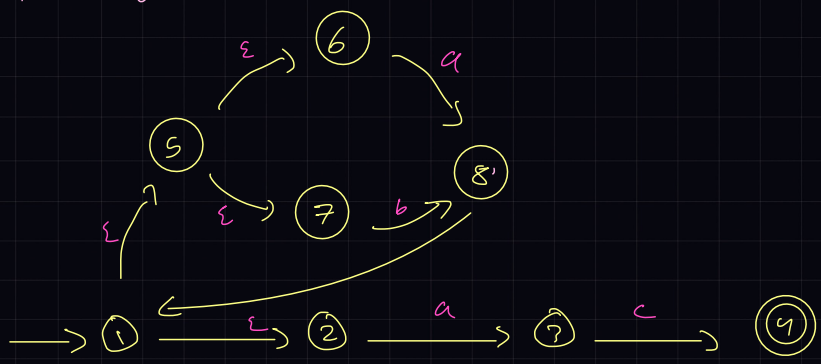
$\rightarrow$ if T $a = 1$

$\rightarrow$ if true $a = 1$

- Draw Parse tree (Leftmost)



(4) answer the following



- Identify the following

$S = \{1, 2, 3, 4, 5, 6, 7, 8\}$

$S_0 = \{1\}$

$\Sigma = \{a, b, c\}$

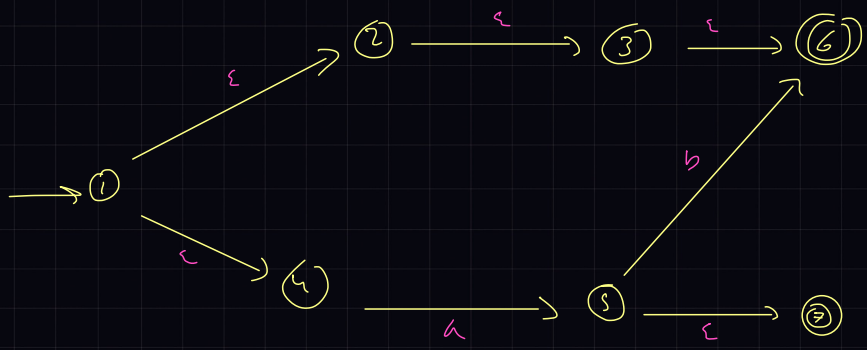
$F = \{4\}$

- Show if abac accept or reject, Show path

Accepted

- Show if aaac accept or reject, Show path

Accepted



NFA	DFA	a
{1, 2, 3, 4, 6}	A	B = {5, 7}

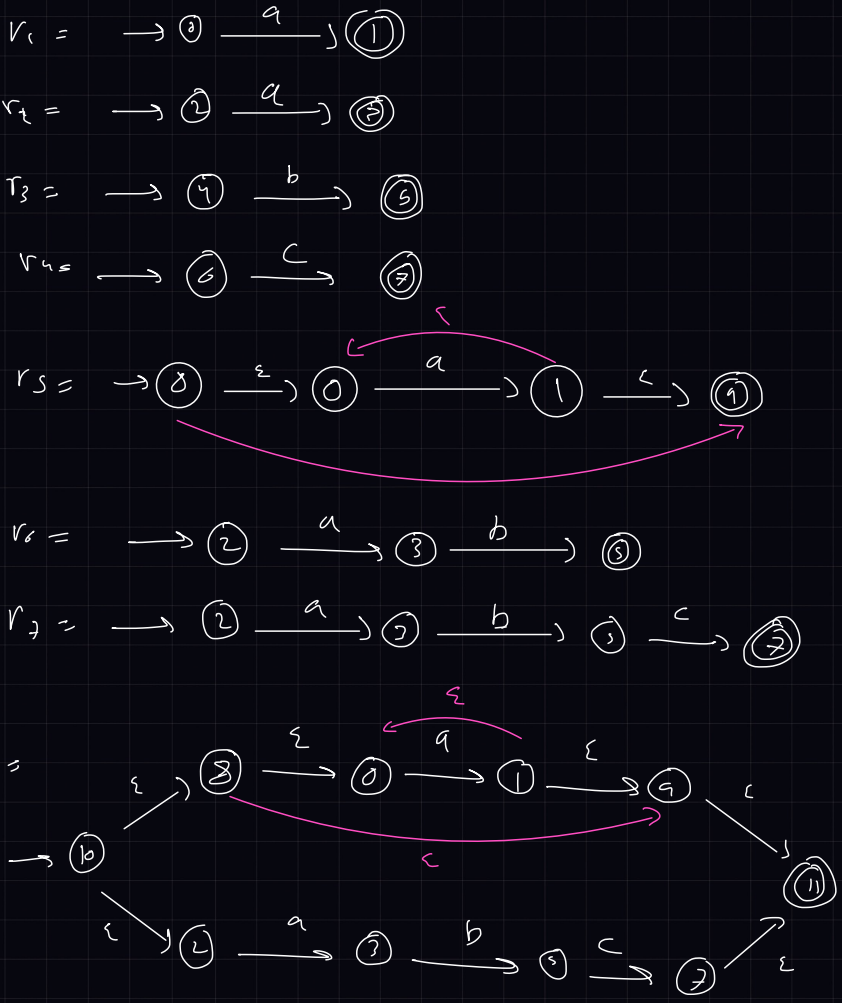
ε - Closure ({1}) = {1, 2, 3, 4, 6} → A

move (A, a) = {5}

ε - closure (move (A, a)) = {5, 7}

(4) Convert from regular expression to NFA

$a^* \mid (ab)c$



Lexems & Tokens

Tokens are the smallest elements of a program that is meaningful to the compiler. They are also known as the fundamental building blocks of the program. Tokens can be classified as follows:

1. Keywords
2. Identifiers
3. Constants
4. Special Symbols
5. Operators
6. Comments
7. Separators

List of Java Keywords

abstract	assert	boolean
break	byte	case
catch	char	class
const	continue	default
do	double	else
enum	exports	extends
final	finally	float
for	goto	if
implements	import	instanceof
int	interface	long
module	native	new
open	opens	package
private	protected	provides
public	requires	return
short	static	strictfp
super	switch	synchronized
this	throw	throws
to	transient	transitive
try	uses	void
volatile	while	with

Example →

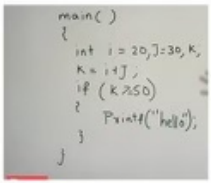
اي keyword اسم التوكين حقها بيكون نفسه

```java
import java.util.Scanner;

class Main {
  public static void main(String[] args) {
    Scanner in = new Scanner(System.in);

    System.out.print("How old are you?: ");
    int age = in.nextInt();

    if (age < 16) {
      System.out.println("Sorry, you are not
      quite old enough to drive!");
    }
    else {
      System.out.println("Yeah! Happy driving!");
    }
  }
}
```

Lexeme:	Token:
import	import
java	identifier
.	seperator
Util	identifier
.	seperator
Scanner	identifier
Class	Class
Main	identifier
{	Left braces
Public	Public
Static	Static
Void	Void
main	identifier
(	left parenthess
String	identifier
[]	Square Brackets
args	identifier
)	right parenthess
{	Left braces
Scanner	identifier
in	identifier
=	equal sign (Assignment)

Lexeme:	Token:
new	new
Scanner	identifier
(	left parenthess
System	identifier
.	seperator
in	identifier
)	right parenthess
;	semicolon
System	identifier
.	seperator
out	identifier
.	seperator
print	identifier (Not Sure, Maybe it's keyword)
(	left parenthess
" How old are you ?: "	Literal

Lexeme:	Token:
)	left parenthess
;	semicolon
int	int
age	identifier
=	equal sign (Assignment)
in	identifier
.	seperator
nextInt()	identifier
;	semicolon
if	if
(	left parenthess
age	identifier
<	RelOp | Less than
16	integer
)	right parenthess
{	Left braces



Symbol table

i

J

K

Lexem	Token
main	main
(	open bracket
)	close bracket
{	curly bracket
int	int
i	identifier
=	equal
20	number
,	comma
J	identifier

Lexem	Token
30	number
K	identifier
;	semicolon
+	plus
if	if
>=	less than or equal
50	number
printf	printf
"hello"	literal
}	curly bracket